

SQLcl project command cheat sheet

Main Commands

config

The option 'LIST' is required and a value must be specified

The `config` command is used to view and manage your project configuration properties.

Usage:

```
project|proj config|cfg {SUBCOMMAND} {OPTIONS}
```

Subcommands:

set | s

Add a new config parameter, or update an existing one.

delete | d

Remove a config parameter from your configuration.

Options:

-list | -li [required]

List your current configuration settings.

Setting	Value (default)
project	test
sqlcl.connectionName	
sqlcl.version	25.2.2.0
schemas	[]
lastReleaseVersion	
export.format.enable	true
export.setTransform.collationClause	NEVER
export.setTransform.constraints	true
export.setTransform.constraints-AsAlter	true
export.setTransform.emitSchema	true
export.setTransform.force	true
export.setTransform.partitioning	false
export.setTransform.segmentAttributes	false
export.setTransform.sizeByteKey-word	true
export.setTransform.sqlterminator	true
export.setTransform.storage	false
export.setTransform.tablespace	false
export.apex.exptype	["READABLE_YAML", "APPLICATION_SOURCE"]
export.apex.expAcIAssignments	false
export.apex.expComments	false
export.apex.skipExportDate	true
export.apex.expirNotif	false
export.apex.expOriginalIds	false
export.apex.expSavedReports	false
export.apex.expPubReports	true
export.apex.expSubscriptions	false
export.apex.expSupportingObjects	N
export.apex.expTranslations	false
export.apex.auditType	NULL
git.protectedBranches	["main", "master"]
git.defaultBranch	main
stage.excludeObjects	["ALL.user"]
stage.generatedFormat	liquibase

-name | -n <name>

The config parameter name can be used when filtering or editing.

-verbose | -v

Show additional details about the current process.

-debug | -de

Enable debug information in the output.

deploy

The option 'FILE' is required and a value must be specified

Deploy the artifact on a target database.

Usage:

```
project|proj deploy|dp {OPTIONS}
```

Options:

-file | -fs <file> [required]

Specify the path to the artifact file.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process.

init

The option 'NAME' is required and a value must be specified

Initialize a new project.

Usage:

```
project|proj init|in {OPTIONS}
```

Options:

-name | -n <name> [required]

Define the project name.

-schemas | -s <schemas>

Specify a comma-separated list of schemas to export, e.g., HR.

-directory | -d <directory> {PATH}

Define the directory in which the project will be created. If not specified, the project will be created in the current directory.

-connection-name | -con <connection-name>

Specify the SQLcl connection name. If not provided, and you are connecting via a named connection, it will automatically add that connection name to the `connectionName` configuration property. If no named connection is used, the `connectionName` property will remain empty.

-makeroot | -mr

Create a root folder named after the project and store all files in this directory.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process.

stage

Generate Liquibase changelogs and changesets for all src and custom SQL files.

Usage:

```
project|proj stage|st {SUBCOMMAND} {OPTIONS}
```

Subcommands:

add-custom | ac

Add a custom file to the stage.

Options:

-branch-name | -bn <branch-name>

Specify the branch name for comparison. If not provided, the value from the configuration file (`git.defaultBranch`) will be used, with "develop" as the default.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process.

export

The command exports database objects, resources, and/or data to your Git repository.

Usage:

```
project|proj export|ex {OPTIONS}
```

Options:

-list | -l

Lists all objects in alphabetical order. The list of objects to export is output based on the current parameters.

-file | -fs <file>

Specify the path to the export file.

-type | -t <type>

Type of resource/data to export (e.g., schema, data, config).

-objects | -o <objects>

Specifies comma-separated list of objects to export. For example, `HR.COUNTRIES`.

-schemas | -s <schemas>

Specifies a comma-separated list of schemas to export. For example, `HR`.

-threads | -t <threads>

Specifies the number of concurrent threads to use with cloned JDBC connections for executing `DBMS_METADATA.getDDL`. The default value is five.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process, in the output..

release

The option 'VERSION' is required and a value must be specified

Move the current set of work into a release state and start a new body of work.

Usage:

```
project|proj release|re {OPTIONS}
```

Options:

-version | -ve <version> [required]

Specify the version to label this release.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process.

verify

Test different aspects of your project, including snapshots, stages, and the overall project.

Usage:

```
project|proj verify|v {SUBCOMMAND} {OPTIONS}
```

Subcommands:

verify-stage | st

Execute all the verify tests registered for the stage group.

snapshot | ss

Execute all the verify tests registered for the snapshot group.

Options:

-group | -gr <group>

Execute a group of verify tests.

-test | -se <test>

Execute a Verification test.

-list-groups | -lig

List all the registered verify service groups.

-list-tests | -lit

If a group is supplied will list all the verify tests in that group, otherwise will list all verify tests registered.

-debug | -de

Enable debug information in the output.

-verbose | -v

Show additional details about the current process.

Help Commands

help project examples

The `project` command supports several subcommands which create a project, export objects from users, generate diffs between versions, generate a release of the changes, generate an artifact with all the changes for the release, and deploy an artifact to a database.

Examples:

Project init command:

```
SQL> project init -name demoproject -schemas hr -verbose
```

Project export command:

```
SQL> project export -schemas hr
```

Project stage command:

```
SQL> project stage -verbose
```

Project release command:

```
SQL> project release -version 1.0
```

Project gen_artifacts command:

```
SQL> project gen_artifacts -name demo -version 1.0 -format zip
```

Project deploy command:

```
SQL> project deploy -file demo_1.0.zip
```

help project init examples

Initialize a project named "apexsb", linked to the "apexsb_data" and "apexsb" schemas, within the directory `/workspace/projects/`, and using the connection name `apexsb_conn`:

```
SQL> project init -name apexsb -schemas apexsb_data,apexsb -d /workspace/projects -con apexsb_conn
```

help project gen-artifact examples

Generate an artifact named **R1-1** in zip format. The artifact is saved in the artifact folder, which will be created if it does not exist

```
SQL> project gen-artifact -name R1 -version 1 -format zip -verbose
```

help project config examples

List all configuration parameters within the project and their values in a table format

```
SQL> project config -list
```

List the configuration for the `sqlcl.version` parameter. Use `-verbose` to include default value and description

```
SQL> project config -list -name sqlcl.version -verbose
```

List all configuration parameters that include the specified name **stage**

```
SQL> project config -list -name stage -verbose
```

help project deploy examples

Extract the artifact file from the specified path, execute `@dist/install.sql`, and initiate the Liquibase installer on the target database

```
SQL> project deploy -file /Users/user1/Desktop/project1/artifact/R1-1.zip -verbose
```

help project export examples

Examples of custom export filters from

```
.dbtools/filters/project.filters
```

```
SQL> project export -debug -list -objects REGIONS,COUNTRIES -schemas HR,SCOTT -threads 4
```

Export a specific Apex application:

```
SQL> project export -objects APEX.100 -schemas APEXUSER
```

help project verify examples

Run verification tests on your project

```
SQL> project verify
```

Execute a specific verification test

```
SQL> project verify -test <name>
```

Execute verification tests for a specified group

```
SQL> project verify -group <name>
```

List all registered verification tests

```
SQL> project verify -list-tests
```

List all registered verification test groups

```
SQL> project verify -list-groups
```

help project stage examples

Create a new directory under **dist/releases/next** based on the current branch name. It adds changelogs and changesets to the folder by diffing files in the current branch and the "from" branch. If no branch specified, it uses the default from the config **stage.branch** parameter

```
SQL> project stage
```

Create a new directory under **dist/releases/next** based on the current branch name. Adds changelogs and changesets by diffing files in the current branch and the "from" branch. Here, the "from" branch is specified as "dev"

```
SQL> project stage -branch-name dev
```

Create a new directory under **dist/releases/next** specifying the branch name

```
SQL> project stage -branch-name branch1
```

help project release examples

To package a unit of work into a release:

```
SQL> project release -version 1.0
```